

Reinforcement Learning - Part 1

MAB, Monte Carlo, and TD Methods

GARP RAI Exam Preparation

What You Will Learn in Part 1:

Topic	Key Concepts Covered
1. RL Fundamentals	Agent, Environment, State, Action, Reward, Policy, Value Functions
2. Multi-Armed Bandit	Exploration vs Exploitation, ϵ -Greedy, Q-Value Updates
3. Monte Carlo Methods	Episode-based Learning, Full Returns, Bias-Variance Trade-off
4. Temporal Difference	Online Learning, Bootstrapping, Q-Learning, SARSA, TD Error

Key to Success: Understand the "exploration vs exploitation" trade-off!

RL - What Makes It Different?

Reinforcement Learning vs Other Paradigms

Paradigm	Key Characteristic
Supervised Learning	Learns from labeled data ; output = prediction/classification
Unsupervised Learning	Finds hidden patterns ; output = clusters/structures
Reinforcement Learning	Learns through trial and error ; output = policy/action

Core RL Concept:

- Agent **interacts** with environment
- Takes **actions**
- Receives **rewards** (positive or negative)
- Goal: **Maximize cumulative future reward**

Analogy - Dog Training:

- Command → Dog takes action → Reward if correct
- Over time, dog learns the **policy**

Exam Trap: RL uses **rewards**, not labels!

The Language of Reinforcement Learning

Term	Definition
Agent	The learner/decision-maker who takes actions
Environment	The world the agent interacts with
Action (a)	A choice the agent can make
State (s)	The current situation of the environment
Reward (R)	Feedback from environment (positive or negative)
Policy (π)	The agent's strategy - maps states to actions
Value Function $V(s)$	How good it is to be in state s
Action-Value $Q(s,a)$	How good it is to take action a in state s

Key Insight:

- Optimal policy: $\pi^*(s) = \arg \max_a Q(s, a)$

Exam Trap: Policy $\neq$ Value Function!

The Mathematics of RL

Expected Future Return (G):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$\gamma = \text{discount factor}$ ($0 \leq \gamma \leq 1$)

State-Value Function $V(s)$:

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

Action-Value Function $Q(s,a)$:

$$Q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

Relationship:

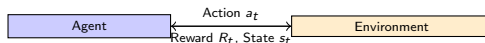
$$V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s, a)$$

$$V^*(s) = \max_a Q^*(s, a)$$

Exam Trap: $V(s)$ = expectation over actions; $Q(s,a)$ = action-specific!

The Agent-Environment Loop

The Interaction Cycle:



Step-by-Step:

- 1 Agent observes **state** s_t
- 2 Agent selects **action** a_t
- 3 Environment returns **reward** R_{t+1} and **next state** s_{t+1}
- 4 Agent updates its knowledge
- 5 Repeat

Formal Sequence:

$$s_0, a_0, R_1, s_1, a_1, R_2, s_2, a_2, R_3, \dots$$

Exam Trap: In MAB, there is only one state!

Real-World RL Applications

AlphaGo / AlphaZero (DeepMind):

- Superhuman performance in Go, Chess, Shogi
- Played **millions** of games against itself
- Discovered **novel strategies**

Robotics:

- Complex manipulation tasks
- Self-driving cars
- Warehouse automation

Business:

- Dynamic Pricing
- Supply Chain Optimization
- Algorithmic Trading
- **RLHF for ChatGPT** (Reinforcement Learning from Human Feedback)

Exam Trap: RLHF is key for LLM fine-tuning!

Key Challenges in RL

1. Sample Efficiency:

- RL requires **millions** of interactions
- Real-world interactions can be expensive or dangerous

2. Reward Design:

- Sparse rewards make learning difficult
- Poor reward design leads to **unintended behaviors**

3. Computational Cost:

- Deep RL requires extensive computational resources
- Training can take days or weeks

4. Safety and Exploration:

- Exploration can be dangerous in real-world
- Agent may take harmful actions while learning

5. Credit Assignment:

- Which actions led to the reward?
- Delayed rewards make this difficult

Exam Trap: Sample inefficiency is RL's biggest challenge!

The Credit Assignment Problem

What is Credit Assignment?

- Determining which actions are responsible for a reward
- In complex tasks, rewards may be **delayed**

Example - Chess:

- A move made 20 steps before checkmate contributed
- How do we know which moves were good?
- The reward (win/loss) comes at the end

Solutions:

- 1 **Monte Carlo**: Uses complete episode returns
- 2 **TD Learning**: Uses one-step or n-step returns
- 3 **Eligibility Traces**: Assigns credit to recent actions

TD(λ) Example:

- $\lambda = 0$: Only immediate action gets credit
- $\lambda = 1$: All actions get credit equally

Exam Trap: Credit assignment is the fundamental problem!

The Exploration-Exploitation Dilemma

Definition:

- **Exploration:** Trying new actions to gather information
- **Exploitation:** Using known actions to maximize reward

Why Balance is Needed:

- Pure exploitation: May miss better options (**regret**)
- Pure exploration: Never uses knowledge (**waste**)
- Optimal: Explore when uncertain, exploit when confident

Strategies:

- 1 **ϵ -Greedy:** Explore with probability ϵ
- 2 **Decaying ϵ :** Explore less over time
- 3 **Thompson Sampling:** Bayesian exploration
- 4 **UCB:** Upper Confidence Bound

UCB Formula:

$$a_t = \arg \max_a \left(Q(a) + c \sqrt{\frac{\ln t}{N(a)}} \right)$$

Exam Trap: UCB uses uncertainty to guide exploration!

Understanding the Discount Factor γ

Why Discount?

- 1 **Uncertainty:** Future rewards are less certain
- 2 **Time Value:** Immediate rewards are more valuable
- 3 **Mathematical Convenience:** Ensures finite returns

Effect of γ :

γ	Behavior	Example
0.0	Myopic, short-term	Immediate profit only
0.5	Moderate short-term	Next few steps matter
0.9	Long-term focus	Future matters significantly
1.0	Infinite horizon	All future steps equal

Example - Two Choices:

- Choice A: $R = 100$ now
- Choice B: $R = 10$ now, $R = 100$ later

With $\gamma = 0.9$:

- Choice A: $G = 100$
- Choice B: $G = 10 + 0.9(100) = 100$ (Tie!)

With $\gamma = 0.5$:

- Choice A: $G = 100$
- Choice B: $G = 10 + 0.5(100) = 60$ (Choice A better!)

Exam Trap: γ controls time preference!

The Multi-Armed Bandit Problem

The Scenario:

- Gambler faces **multiple slot machines**
- Each machine has **unknown** reward distribution
- Goal: **Maximize total winnings**

Key MAB Characteristics:

- 1 **Single state** - environment doesn't change
- 2 Actions don't affect future states
- 3 Each action gives **immediate reward**
- 4 No long-term consequences

The Core Dilemma:

- **Exploit**: Play best machine so far
- **Explore**: Try other machines to find better ones

MAB vs General RL:

- MAB: Single state, no future consequences
- General RL: Multiple states, actions affect future

Exam Trap: MAB has **ONLY ONE STATE!**

MAB - The Three Strategies

Three Strategies for MAB

Strategy 1: Greedy (Pure Exploitation)

- Always choose **highest Q-value**
- Pros: Maximizes immediate reward
- Cons: May miss better machine

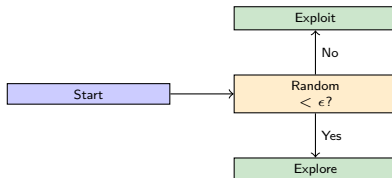
Strategy 2: Random (Pure Exploration)

- Always choose **random** machine
- Pros: Gathers information about all machines
- Cons: Never uses knowledge

Strategy 3: ϵ -Greedy (Balanced)

- With probability ϵ : **Explore** (random)
- With probability $1 - \epsilon$: **Exploit** (best)
- **Balances** exploration and exploitation

Decision Process:



Exam Trap: ϵ -greedy with decay is best practical!

Q-Value = Average of Observed Rewards

$$Q_k^n = \frac{1}{n} \sum_{j=1}^n R_j$$

Incremental Update:

$$Q_k^n = Q_k^{n-1} + \frac{1}{n}(R_n - Q_k^{n-1})$$

Example: Machine A played 5 times:

- Rewards: [1.2, 0.8, 1.5, 0.3, 1.1]
- Sum = 1.2 + 0.8 + 1.5 + 0.3 + 1.1 = 4.9
- $Q(A) = 4.9 / 5 = 0.98$

Step-by-Step:

- 1 Play 1: R=1.2, Q=1.2
- 2 Play 2: R=0.8, Q=(1.2+0.8)/2=1.0
- 3 Play 3: R=1.5, Q=(1.2+0.8+1.5)/3=1.167
- 4 Play 4: R=0.3, Q=(1.2+0.8+1.5+0.3)/4=0.95
- 5 Play 5: R=1.1, Q=(1.2+0.8+1.5+0.3+1.1)/5=0.98

Exam Trap: Q-values are **averages**, not sums!

How ϵ Decays Over Time

$$\epsilon_t = \beta^{t-1}$$

where β = decay factor ($0 < \beta < 1$)

Example with $\beta = 0.85$:

Trial	ϵ_t	Decision
1	1.00	Always Explore
2	0.85	85% Explore
5	0.522	52.2% Explore
10	0.232	23.2% Explore
20	0.046	4.6% Explore
50	0.0004	Pure Exploit

Key Insight:

- High ϵ early = More exploration
- Low ϵ later = More exploitation
- When ϵ approaches 0, agent is fully exploiting

Linear Decay Alternative:

$$\epsilon_t = \epsilon_{max} - \frac{t}{T}(\epsilon_{max} - \epsilon_{min})$$

Exam Trap: $\epsilon_t = \beta^{t-1}$, not β^t !

MAB - Worked Example

MAB - Step-by-Step Example

Setup:

- 3 machines: A, B, C
- True means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\beta = 0.85$

Trial 1: $\epsilon = 1.00$, random=0.7 $\rightarrow$ Explore

- Choose A, Reward=1.2, $Q(A)=1.2$

Trial 2: $\epsilon = 0.85$, random=0.99 $\rightarrow$ Exploit

- Choose A (best), Reward=0.8, $Q(A)=1.0$

Trial 3: $\epsilon = 0.7225$, random=0.65 $\rightarrow$ Explore

- Choose B, Reward=1.7, $Q(B)=1.7$

Trial 4: $\epsilon = 0.614$, random=0.50 $\rightarrow$ Explore

- Choose C, Reward=0.6, $Q(C)=0.6$

Trial 5: $\epsilon = 0.522$, random=0.70 $\rightarrow$ Exploit

- Best: B ($Q=1.7$), Reward=0.5, $Q(B)=1.1$

Machine	Q-Value
A	0.80
B	0.50
C	0.70

After 10 Trials:

Exam Trap: Estimates may differ from true means due to sampling error!

The Risk of Pure Exploitation

Scenario:

- 3 machines after 5 plays:
 - $Q(A) = 1.5$ (from 5 lucky plays)
 - $Q(B) = 0.0$ (never played)
 - $Q(C) = 0.0$ (never played)
- True means: $\mu_A = 0.2$, $\mu_B = 2.0$, $\mu_C = 1.8$

Greedy Strategy:

- Always plays A ($Q=1.5$)
- Expected reward = 0.2 per play
- Total reward = $0.2 \times 1000 = 200$

Optimal Strategy:

- Always plays B (true mean = 2.0)
- Total reward = $2.0 \times 1000 = 2000$

Regret:

- Regret = $2000 - 200 = 1800$ (huge loss!)

Lesson:

- Greedy got **stuck** with suboptimal machine
- **Exploration** would have discovered better machine

Exam Trap: Greedy can be catastrophically suboptimal!

MAB - Random Strategy Problem

The Problem of Pure Exploration

Scenario:

- 3 machines: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- 1000 plays

Random Strategy:

- Expected reward = average of all means
- $= (0.8 + 0.5 + 0.7) / 3 = 0.667$
- Total reward = $0.667 \times 1000 = 667$

Optimal Strategy:

- Always plays A (0.8)
- Total = $0.8 \times 1000 = 800$

Comparison:

Strategy	Expected Reward
Optimal	800
ϵ -greedy (0.1)	787
Greedy (if lucky)	800
Greedy (if unlucky)	200
Random	667

The Problem:

- Random strategy **wastes** knowledge
- After 1000 plays, knows best machine
- But continues to play random machines!

Exam Trap: Random **never exploits** - wastes knowledge!

ϵ -Greedy - The Balanced Approach

Setup:

- 3 machines: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\epsilon = 0.10$ (constant)
- 1000 plays

Expected Behavior:

- 90% plays: best machine (A) $\rightarrow$ reward 0.8
- 10% plays: random $\rightarrow$ average 0.667
- Expected reward = $0.9(0.8) + 0.1(0.667) = 0.787$
- Total ≈ 787

With Decay:

- Early: More exploration to find best machine
- Late: More exploitation to maximize reward
- Often **outperforms** constant ϵ

Why It Works:

- Balances exploration and exploitation
- Learns from experience
- Adapts over time

Exam Trap: ϵ -greedy with decay is usually better!

Choosing the Decay Factor β

Effect of Different β :

Trial	$\beta = 0.85$	$\beta = 0.95$	$\beta = 0.99$
1	1.00	1.00	1.00
5	0.522	0.815	0.961
10	0.232	0.630	0.914
20	0.046	0.377	0.827
50	0.0004	0.077	0.606
100	≈ 0	0.006	0.366
200	≈ 0	0.00003	0.134

Interpretation:

- $\beta = 0.85$: Fast decay, quick exploitation
 - Good for simple problems
- $\beta = 0.95$: Moderate decay, balanced
 - Good for most practical problems
- $\beta = 0.99$: Slow decay, long exploration
 - Good for complex problems

When ϵ Becomes Very Small:

- $\beta = 0.85$: 57 trials
- $\beta = 0.95$: 230 trials
- $\beta = 0.99$: 2300 trials

Exam Trap: Smaller β = faster decay = less exploration later!

MAB - Thompson Sampling

Thompson Sampling - A Bayesian Alternative

The Concept:

- Maintain **probability distribution** for each machine
- Sample from each distribution
- Choose **maximum sampled value**
- Naturally balances exploration and exploitation

How It Works:

- 1 Each machine has distribution (Beta for binary, Normal for continuous)
- 2 Parameters update with each observation (Bayesian updating)
- 3 Each trial:
 - 1 Sample value from each machine's distribution
 - 2 Choose machine with highest sampled value
 - 3 Observe reward and update distribution

Example - Binary Rewards:

- A: Beta(5,2) $\rightarrow$ mean=5/7=0.714
- B: Beta(2,3) $\rightarrow$ mean=2/5=0.400
- C: Beta(3,2) $\rightarrow$ mean=3/5=0.600
- Sample: A=0.72, B=0.35, C=0.68 $\rightarrow$ Choose A

Advantages:

- No ϵ parameter to tune
- Balances based on uncertainty
- Theoretically optimal (Bayesian)

Exam Trap: Thompson Sampling is Bayesian!

MAB - Regret Analysis

Regret - Measuring Performance

$$\text{Regret}(T) = T \times \mu^* - \sum_{t=1}^T \mu_{a_t}$$

Interpretation:

- Opportunity cost of not knowing the best machine
- Cumulative loss from suboptimal choices
- Goal: **Minimize regret**

Example:

- True means: $\mu_A = 0.8$, $\mu_B = 0.5$, $\mu_C = 0.7$
- $\mu^* = 0.8$ (Machine A)
- If agent plays Machine B 100 times:
 - Regret per play = $0.8 - 0.5 = 0.3$
 - Total regret = $0.3 \times 100 = 30$

Expected Regret (T=1000):

Strategy	Expected Regret
Optimal	0
ϵ -greedy (0.1)	13
Greedy (unlucky)	600
Random	133

Key Insight:

- Good algorithms achieve **sublinear regret**
- ϵ -greedy with decay achieves $O(\log T)$ regret

Exam Trap: Regret is cumulative loss - minimize it!

MAB - A/B Testing Application

A/B Testing as a Multi-Armed Bandit

Traditional A/B Testing:

- Split traffic 50/50 between options
- Test for statistical significance
- Then implement the winner
- Problem: **Wastes traffic** on inferior options

MAB Approach:

- Each variant = **slot machine**
- Use ϵ -greedy or Thompson Sampling
- Dynamically allocate more traffic to better variants

Example - Ad Click-Through Rates:

- Variant A: 5% CTR (true)
- Variant B: 3% CTR (true)
- Variant C: 4% CTR (true)

Traditional (100,000 impressions):

- Each variant gets 33,333 impressions
- Expected clicks = $0.05(33,333) + 0.03(33,333) + 0.04(33,333) = 4,000$

MAB Approach:

- Early: Explore all variants
- Later: 90% to best (A)
- Expected clicks $\approx 4,850$
- **21% more clicks!**

Exam Trap: MAB significantly outperforms A/B testing!

MAB - Key Takeaways

Key Concepts:

1 MAB Characteristics:

- Single state, immediate rewards, no future consequences

2 Three Strategies:

- Greedy = Pure exploitation
- Random = Pure exploration
- ϵ -Greedy = Balanced (best)

3 Q-Value:

$$Q_k^n = \frac{1}{n} \sum R_j$$

4 ϵ Decay:

$$\epsilon_t = \beta^{t-1}$$

5 Regret:

- Loss from not knowing the optimal action

6 Thompson Sampling:

- Bayesian approach, no ϵ parameter

Exam Success: MAB is the foundation - master it!

Monte Carlo - Learning from Complete Episodes

What is Monte Carlo RL?

- Learns from **complete episodes**
- Episode = full sequence from start to terminal state
- Example: Complete game of chess

Key Characteristics:

- **Wait until episode ends** to update
- Uses **actual returns** (G_t)
- **Unbiased** - learns from real experience

MC vs MAB:

- MAB: Each trial independent, one-step
- MC: Multiple steps, actions affect future states

When to Use MC:

- Tasks with **defined terminal states** (episodic)
- When you want **unbiased** estimates
- When episodes are not too long

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

Exam Trap: MC requires **complete episodes**!

What is an Episode?

Definition:

- Complete trajectory from start to terminal state
- Sequence: $S_0, A_0, R_1, S_1, A_1, \dots, S_T$
- Terminal state S_T ends the episode

Examples:

- 1 **Nim**: Start $\rightarrow$ coin removal $\rightarrow$ last coin wins
- 2 **Chess**: Opening $\rightarrow$ moves $\rightarrow$ Checkmate
- 3 **Board Game**: Start $\rightarrow$ moves $\rightarrow$ Win/Lose

Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

Nim Example:

- Episode length: 3 steps (if Player 1 wins)
- Step 1: Pick 3 coins ($S=6 \rightarrow S=3$)
- Step 2: Opponent picks 2 coins ($S=3 \rightarrow S=1$)
- Step 3: Player 1 picks 1 coin ($S=1 \rightarrow S=0$, Win!)

Update Only at Episode End:

- No updates within episode
- All state-action pairs updated at end

Exam Trap: MC updates **only at episode end!**

Calculating Returns in MC

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

Example - Nim with $\gamma = 1$:

- Episode: $S=6 \rightarrow S=3 \rightarrow S=1 \rightarrow S=0$ (Win)
- Rewards: $R_1 = 0, R_2 = 0, R_3 = 1$
- For step 1 ($t=0$): $G_0 = 0 + 0 + 1 = 1$
- For step 2 ($t=1$): $G_1 = 0 + 1 = 1$
- For step 3 ($t=2$): $G_2 = 1$

Example - Nim with $\gamma = 0.9$:

- $G_0 = R_1 + 0.9R_2 + 0.9^2R_3 = 0 + 0 + 0.81 = 0.81$
- $G_1 = R_2 + 0.9R_3 = 0 + 0.9 = 0.9$
- $G_2 = R_3 = 1$

Why Discount?

- Future rewards are less certain
- Discounting gives more weight to immediate rewards
- γ close to 0: Short-term focus
- γ close to 1: Long-term focus

Exam Trap: MC uses actual return G_t !

Monte Carlo - First-Visit vs Every-Visit

First-Visit vs Every-Visit MC

The Distinction:

- In an episode, same state-action pair (s, a) may be visited multiple times
- Two ways to handle this:

First-Visit MC:

- Track which (s, a) have been seen
- Only update $Q(s,a)$ the **first time** it occurs
- Uses return from that first occurrence
- **Unbiased** estimate of $Q(s,a)$

Every-Visit MC:

- Update $Q(s,a)$ **every time** it occurs
- Uses return from each occurrence
- **Slightly biased** but often lower variance

Example:

- Episode: $(S1,A1)$, $(S2,A2)$, $(S1,A1)$, $(S3,A3)$, Terminal
- First-visit: Update $(S1,A1)$ once (using first occurrence)
- Every-visit: Update $(S1,A1)$ twice (both occurrences)

When to Use:

- First-visit: Theoretical purity, unbiased estimates
- Every-visit: More sample-efficient, practical use

Exam Trap: First-visit = unbiased; Every-visit = lower variance!

MC NIM - First Episode

Setup:

- 6 coins, remove 1, 2, or 3
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly (not optimally)

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1$, $A=1 \rightarrow S'=0$ (Player 1 wins!)
- 3 Return: $G = 1$ (win)

MC Updates (Episode End):

- For (6,3): $Q(6, 3) = 0 + 0.05(1 - 0) = 0.05$
- For (1,1): $Q(1, 1) = 0 + 0.05(1 - 0) = 0.05$

After Episode 1:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0.05

Exam Trap: MC updates all pairs in episode with same G!

MC NIM - Second Episode

Episode 2:

- 1 $S=6, A=1 \rightarrow S'=5$ (opponent picks 1)
- 2 $S=4, A=3 \rightarrow S'=1$ (opponent picks 1)
- 3 $S=0$ (Player 1 loses!)
- 4 Return: $G = -1$ (loss)

MC Updates (Episode End):

- For (6,1): $Q(6, 1) = 0 + 0.05(-1 - 0) = -0.05$
- For (4,3): $Q(4, 3) = 0 + 0.05(-1 - 0) = -0.05$

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	-0.05
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	-0.05	0	0.05

Exam Trap: Negative G updates Q-values downward!

MC NIM - Third Episode

Episode 3:

- 1 $S=6, A=3 \rightarrow S'=3$ (opponent picks 2)
- 2 $S=1, A=1 \rightarrow S'=0$ (Player 1 wins!)
- 3 Return: $G = 1$

MC Updates:

- $Q(6,3) = 0.05 + 0.05(1 - 0.05) = 0.0975$
- $Q(1,1) = 0.05 + 0.05(1 - 0.05) = 0.0975$

After Episode 3:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.0975	0	0	0	0	-0.05
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	-0.05	0	0.0975

Exam Trap: Q-values gradually move toward true values!

MC NIM - After 1,000,000 Episodes

Final Q-Table:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.071	0.380	0	0.774
Pick 2	0	1	-1	-0.110	0	1
Pick 3	0	0	1	-0.858	0	0.020

Optimal Strategy:

- S=6: Pick 2 (Q=1)
- S=4: Pick 1 (Q=0.380)
- S=3: Pick 3 (Q=1)
- S=2: Pick 2 (Q=1)
- S=1: Pick 1 (Q=1)

Interpretation:

- Q=1 = winning action
- Q=-1 = losing action
- Optimal strategy ensures win

Exam Trap: Q=1 = winning action; Q=-1 = losing action!

Monte Carlo - Advantages and Disadvantages

MC - Pros and Cons

Advantages:

- 1 Unbiased estimates - uses actual returns
- 2 Simple to understand and implement
- 3 Works well with long episodes
- 4 No bootstrapping - no risk of propagating errors

Disadvantages:

- 1 High variance - returns can fluctuate significantly
- 2 Requires complete episodes - cannot handle continuous tasks
- 3 Slow learning - must wait until episode end
- 4 Inefficient for long episodes - updates are infrequent

When to Use MC:

- Tasks with clear terminal states (episodic)
- When you can run many episodes
- When you want unbiased value estimates

Comparison:

MC	TD
Unbiased	Biased
High variance	Low variance
Episode-end	Step-by-step
Episodic only	Episodic + Continuous

Exam Trap: MC = unbiased but high variance!

Monte Carlo - Key Takeaways

Key Concepts:

1 Definition:

- Learns from complete episodes
- Updates only at episode end
- Uses actual returns (G_t)

2 Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t - Q(s, a))$$

3 Return Calculation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

4 First-Visit vs Every-Visit:

- First-visit: unbiased
- Every-visit: lower variance

5 Requirements:

- **Must have** terminal states (episodic tasks)
- Cannot handle continuous tasks

6 Trade-off:

- **Unbiased** but **high variance**

Exam Success: MC = episodic tasks only!

Temporal Difference - Learning Step by Step

What is TD Learning?

- Learns from **incomplete episodes**
- Updates **after each step** (online learning)
- Uses **bootstrapping** - estimates of future values

Key Characteristic:

- **Doesn't wait** for episode end
- Updates using $R_{t+1} + \gamma V(s')$
- **Biased** but **lower variance**

TD vs MC:

MC	TD
Episode-end updates	Step-by-step updates
Uses G_t (actual return)	Uses $R_{t+1} + \gamma V(s')$ (estimate)
Unbiased	Biased (bootstrapping)
High variance	Low variance
Episodic only	Episodic + Continuous

TD Update Formula:

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

= the "**surprise**"

Exam Trap: TD uses bootstrapping!

TD - Bootstrapping Explained

Bootstrapping - The Core of TD

What is Bootstrapping?

- Using a **current estimate** to update another estimate
- "Pulling yourself up by your own bootstraps"
- Instead of waiting for final outcome, use current best guess

MC vs TD Bootstrapping:

- **MC**: No bootstrapping - uses actual returns
- **TD**: Bootstrapping - uses estimates of future values

Example - Nim:

- $S=6$, take action that leads to $S'=1$
- MC: Wait until episode end to know G
- TD: Use $V(1)$ (current estimate) + immediate reward

TD Target:

$$R_{t+1} + \gamma V(s')$$

- Better guess than $V(s)$ alone
- Uses immediate reward + estimate of future

Why Bootstrapping Works:

- Updates can be made immediately
- Doesn't require complete episodes
- Often converges faster than MC

TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

- Positive: State better than thought
- Negative: State worse than thought
- Zero: Matches current estimate

Exam Trap: Bootstrapping introduces bias but reduces variance!

Q-Learning - The Most Popular TD Algorithm

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Key Features:

- ① **Off-policy:** Learns optimal policy regardless of behavior
- ② **Model-free:** No environment model needed
- ③ **TD method:** Uses bootstrapping

Why Off-Policy?

- Uses $\max_{a'} Q(s', a')$ (best possible action)
- Doesn't matter which action was actually taken
- Learns about **optimal policy**

Algorithm Steps:

- ① Initialize $Q(s,a)$ arbitrarily
- ② For each episode:
 - ① Initialize state s
 - ② For each step:
 - ① Choose a from s using ϵ -greedy
 - ② Take action a , observe R and s'
 - ③ Update $Q(s,a)$ using TD target
 - ④ $s \leftarrow s'$

Convergence Properties:

- Converges to optimal Q^* with probability 1
- Requires all state-action pairs visited infinitely often

Exam Trap: Q-learning is off-policy!

SARSA - On-Policy TD Learning

Update Formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R_{t+1} + \gamma Q(s', a') - Q(s, a))$$

where a' is the **actual action** taken in s'

Key Features:

- ① **On-policy:** Learns the policy being followed
- ② **Model-free:** No environment model needed
- ③ **TD method:** Uses bootstrapping

Why On-Policy?

- Uses $Q(s', a')$ where a' is actual next action
- Learns about **current policy**, not optimal
- Policy and learning are linked

SARSA vs Q-Learning:

Q-Learning	SARSA
Off-policy	On-policy
Uses $\max_{a'} Q(s', a')$	Uses $Q(s', a')$ (actual)
More aggressive	More conservative
Learns optimal	Learns current

Algorithm Steps:

- ① Initialize $Q(s, a)$
- ② For each episode:
 - ① Choose a from s using policy
 - ② For each step:
 - ① Take action, observe R, s'
 - ② Choose a' from s' using policy
 - ③ Update $Q(s, a)$ using $Q(s', a')$
 - ④ $s \leftarrow s', a \leftarrow a'$

Exam Trap: SARSA is on-policy; Q-learning is off-policy!

TD - Q-Learning vs SARSA Comparison

Q-Learning vs SARSA - Detailed Comparison

Key Difference:

- Q-learning: $R + \gamma \max_{a'} Q(s', a')$
- SARSA: $R + \gamma Q(s', a')$

Behavioral Differences:

Q-Learning	SARSA
Best possible next action	Actual next action
More aggressive	More conservative
Learns risky policies	Learns safe policies
May choose danger if avoided optimally	May avoid danger under current policy

Example - Cliff Walking:

- Agent must walk along cliff to reach goal
- Falling off cliff = large negative reward

Q-Learning:

- Learns optimal path (right next to cliff)
- Risky if agent sometimes makes mistakes

SARSA:

- Learns safer path (further from cliff)
- Accounts for suboptimal actions

Which to Use?

- Q-learning: Want optimal policy, can explore safely
- SARSA: Safety important, exploration is risky

Exam Trap: Q-learning = aggressive; SARSA = conservative!

TD - NIM Example Step 1

TD NIM - First Episode

Setup:

- 6 coins, remove 1, 2, or 3
- $\alpha = 0.05$, $\gamma = 1$
- Opponent plays randomly

Episode 1:

- 1 $S=6$, $A=3 \rightarrow S'=3$, $R=0$
- 2 $S=1$, $A=1 \rightarrow S'=0$, $R=1$ (Win!)

TD Updates (Step by Step):

- **Step 1:** $S=6$, $A=3$, $S'=3$, $R=0$
 - $V(3) = 0$ (current estimate)
 - TD target $= 0 + 1(0) = 0$
 - $Q(6,3) = 0 + 0.05(0 - 0) = 0$
- **Step 2:** $S=1$, $A=1$, $S'=0$, $R=1$
 - $V(0) = 0$ (terminal state)
 - TD target $= 1 + 1(0) = 1$
 - $Q(1,1) = 0 + 0.05(1 - 0) = 0.05$
- Update $V(1)$: $V(1) = \max Q(1,a) = 0.05$

After Episode 1:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0	0	0

Exam Trap: TD updates within episode using $V(s')$!

TD NIM - Second Episode

Episode 2:

- ① $S=6, A=1 \rightarrow S'=5, R=0$
- ② $S=4, A=3 \rightarrow S'=1, R=0$
- ③ $S=0$ (Player 1 loses!)

TD Updates:

- **Step 1:** $S=6, A=1, S'=5, R=0$
 - $V(5) = 0$
 - TD target $= 0 + 1(0) = 0$
 - $Q(6,1) = 0 + 0.05(0 - 0) = 0$
- **Step 2:** $S=4, A=3, S'=1, R=0$
 - $V(1) = 0.05$ (from Episode 1)
 - TD target $= 0 + 1(0.05) = 0.05$
 - $Q(4,3) = 0 + 0.05(0.05 - 0) = 0.0025$

After Episode 2:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.05	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0.0025	0	0

$$V(1) = 0.05, V(4) = 0.0025$$

Exam Trap: TD uses $V(1)$ from previous episode!

TD NIM - Third Episode

Episode 3:

- ① $S=6, A=3 \rightarrow S'=1, R=0$
- ② $S=1, A=1 \rightarrow S'=0, R=1$ (Win!)

TD Updates:

- **Step 1:** $S=6, A=3, S'=1, R=0$
 - $V(1) = 0.05$ (from Episode 1)
 - TD target $= 0 + 1(0.05) = 0.05$
 - $Q(6,3) = 0 + 0.05(0.05 - 0) = 0.0025$
- **Step 2:** $S=1, A=1, S'=0, R=1$
 - TD target $= 1 + 1(0) = 1$
 - $Q(1,1) = 0.05 + 0.05(1 - 0.05) = 0.0975$

After Episode 3:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	0.0975	0	0	0	0	0
Pick 2	0	0	0	0	0	0
Pick 3	0	0	0	0.0025	0	0.0025

Exam Trap: TD values propagate backward through episode!

TD NIM - After 1,000,000 Episodes

Final Q-Table:

Action	S=1	S=2	S=3	S=4	S=5	S=6
Pick 1	1	-1	-0.105	0.383	0	0.845
Pick 2	0	1	-1	0.058	0	1
Pick 3	0	0	1	-0.842	0	0.011

Optimal Strategy:

- S=6: Pick 2
- S=4: Pick 1
- S=3: Pick 3
- S=2: Pick 2
- S=1: Pick 1

Key Insight:

- Both MC and TD converge to **same optimal strategy**
- TD learns faster than MC

Exam Trap: Both MC and TD converge to optimal policy!

TD - Advantages and Disadvantages

TD - Pros and Cons

Advantages:

- 1 Can learn online - updates after each step
- 2 Works for continuous tasks
- 3 Lower variance than MC
- 4 More sample efficient
- 5 Often converges faster

Disadvantages:

- 1 Biased - uses estimates
- 2 May propagate errors
- 3 Requires careful tuning
- 4 Can be unstable

When to Use TD:

- Continuous (non-episodic) tasks
- When you need online learning
- When episodes are very long
- When you want faster learning

Comparison:

MC	TD
Unbiased	Biased
High variance	Low variance
Episode-end	Step-by-step
Episodic only	Episodic + Continuous

Exam Trap: TD = biased but lower variance!

TD - Key Takeaways (Part 1)

Key Concepts:

1 TD Learning:

- Learns online, updates after each step
- Uses bootstrapping

2 TD Update:

$$V(s) \leftarrow V(s) + \alpha(R_{t+1} + \gamma V(s') - V(s))$$

3 TD Error:

$$\delta = R_{t+1} + \gamma V(s') - V(s)$$

4 Q-Learning:

- Off-policy
- Uses $\max_{a'} Q(s', a')$
- More aggressive

5 SARSA:

- On-policy
- Uses $Q(s', a')$ (actual action)
- More conservative

Exam Success: Understand the bias-variance trade-off!

TD - TD(λ) Introduction

TD(λ) - Bridging MC and TD

The Idea:

- MC = TD(1) - uses full return
- TD(0) = One-step TD
- TD(λ) - combines many n-step returns

n-Step Returns:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$$

- n=1: TD(0) target
- n= ∞ : MC target

λ -Return:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- $\lambda = 0$: TD(0)
- $\lambda = 1$: MC
- $0 < \lambda < 1$: Mix of n-step returns

Intuition:

- λ controls how far into future you look
- Lower λ : More like TD (lower variance, higher bias)
- Higher λ : More like MC (higher variance, lower bias)

TD(λ) Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G_t^\lambda - Q(s, a))$$

Exam Trap: $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$!

TD - Complete Summary

Key Concepts:

1 TD Learning:

- Learns online, updates after each step
- Uses bootstrapping

2 Q-Learning:

- Off-policy
- Uses $\max_{a'} Q(s', a')$
- More aggressive

3 SARSA:

- On-policy
- Uses $Q(s', a')$
- More conservative

4 TD(λ):

- $\lambda = 0 = \text{TD}$, $\lambda = 1 = \text{MC}$
- Can be more efficient than TD(0)

5 TD vs MC:

- TD: Biased, lower variance, online
- MC: Unbiased, higher variance, episode-end

Exam Success: Understand the bias-variance trade-off between MC and TD!

Part 1 - Complete Summary

What We Covered:

① RL Fundamentals:

- Agent, Environment, Actions, States, Rewards
- Policy, Value Functions, Discount Factor

② Multi-Armed Bandit:

- Exploration vs Exploitation
- ϵ -Greedy, Q-Value Updates
- Regret Analysis

③ Monte Carlo Methods:

- Episode-based Learning
- Full Returns, First-Visit vs Every-Visit
- NIM Game Example

④ Temporal Difference:

- Bootstrapping, Online Learning
- Q-Learning, SARSA
- $TD(\lambda)$

Key Insight: MC = unbiased but high variance; TD = biased but low variance!